SEA × OPENAI CODEX HACKATHON · VIETNAM 2026

Monastack

Mono + **AI** + **Stack** = **one AI-native company stack**

A workspace where **people and AI agents run a company together**: clean data by design, agents you can talk to like coworkers, and intelligence that runs at the edge.

- AI-Native Products & Operations
- Autonomous & Adaptive AI
- Deep Domain AI

Agents can now rebuild almost any app.
So why are we rebuilding apps made for humans?

github.com/morluto/rea ● ~408k ★

"Reverse Engineer Anything": **one MCP server for reverse engineering binaries, apps and runtime behavior**. Agents can take apart well-known software and rebuild similar features.

It's one of the most talked-about repos in AI right now, and it deserves the attention.

Our take: it's a great party trick

- A recreated app still has a **human interface**: menus, tabs, dashboards, settings pages.
- The next big user of software is **an agent acting on our behalf**, not a person clicking.
- Cloning yesterday's UI gives the agent the same mess it already struggles with.

Why clone the past when the user itself is changing?

INSIGHT

Humans and agents don't see the same company

	A HUMAN SEES	AN AI AGENT SEES
Input	Pixels, layouts, colors, icons	**Tokens, records, schemas**
Navigation	Menus, tabs, 12 SaaS logins	**Tool calls (MCP) with scoped permissions**
Noise	Skims past it without thinking	**Every token can sway the answer**
Memory	Years of tacit company context	**Only what fits in the context window**
Contradictions	"Ask Mai, she knows which sheet is right"	**Picks one, or blends both, with full confidence**
Working hours	8 hours, one task at a time	**24/7, many tasks in parallel**

Agents will soon run largely on their own, following our direction. When they do, **the UI layer of every SaaS becomes a translation layer nobody needs.** The data layer becomes the product.

PROBLEM 1 · FRAGMENTATION

A typical company runs on a dozen tools, and each one has its own version of the truth

Helpdesk tickets, chats	**Warehouse** stock, orders	**Git & CI** issues, PRs	**App** events, crashes	**HRM** people, leave	**CRM** leads, deals
Accounting journals, invoices	**Chat** threads, DMs	**Docs** wikis, files	**Calendar** meetings	**Spreadsheets** "final_v3.xlsx"	**Email** everything else

The trend

Teams are pulling everything into **one project** (monorepos, single workspaces) so it's easier to manage and **so AI can read it all**.

The catch

Pouring every silo into one place **moves the mess without cleaning it**. Twelve data models, duplicate customers and conflicting records end up in the same context window.

PROBLEM 2 · CONTEXT QUALITY

Messy context makes agents confidently wrong

01
Noise
Logs, signatures and UI clutter bury the facts that matter.

02
Context poisoning
One wrong fact or earlier hallucination gets reused as truth.

03
Overload
Too much context, so the model fixates on history and misses the task.

04
Confusion
Irrelevant tools and documents push it toward the wrong action.

05
Clash
CRM says "active", billing says "churned". Which one is right?

06
Staleness
Old records with no version or timestamp look current.

07
Duplicates
"ABC Co.", "Cty ABC", "abc corp": is that three customers or one?

08
Leakage
The agent reads data its requester was never allowed to see.

09
Prompt injection
Instructions hidden inside tickets, emails and documents.

10
Context rot
Recall drops in long contexts (the "lost in the middle" effect).

● **Garbage in → confident garbage out**

The result is **hallucinations, bad decisions and unsafe actions**, and a bigger model doesn't fix bad inputs.

OUR ANSWER

Monastack: rebuild the foundation for agents, not the apps

One AI-native stack, designed in from day one: simpler, cleaner, and easy for both agents and people to work with.

PILLAR 1

Clean by construction

Every source writes through **standard schemas** at the gate. Data is typed, deduplicated, versioned and permission-scoped before any agent reads it.

PILLAR 2

Agents as coworkers

Agents have identities, desks, voices and an auditable work history. You **talk to them like colleagues** in a virtual office instead of shuffling cards on a board.

PILLAR 3

Edge-first mesh

Local AI on every site, peer data nodes with replicas, and **frontier LLM nodes** used only when a task needs them. The result is lower server cost, privacy and offline resilience.

...all built on **proven open source wrapped with MCP**, instead of rewritten from scratch.

DON'T REBUILD: ORCHESTRATE

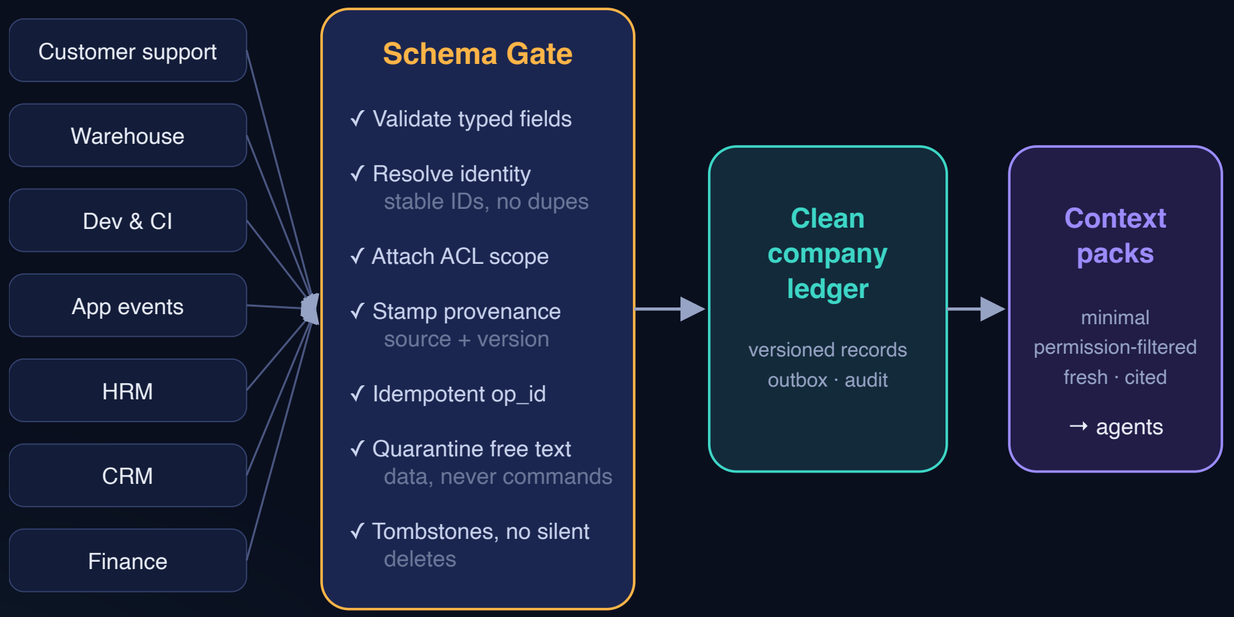
Proven open source underneath, MCP on top

CAPABILITY	OPEN SOURCE WE STAND ON	MONASTACK LAYER (SCHEMA + MCP)	STATUS
Chat & federation	**Matrix** (Synapse / Conduit)	Room policies, agent identities, `rooms.read / post`	PLAN
Voice & video	**WebRTC** + **LiveKit**	Policy-gated tokens, room-epoch isolation	TESTED
Customer support	**Chatwoot**	`support.case` schema, CS-agent tools	PLAN
CRM	**Twenty**	`customer / opportunity / contract`	PLAN
ERP · inventory · HRM · accounting	**Odoo** / **ERPNext**	`stock · leave · expense · journal`	PLAN
Dev & code	**Forgejo / Gitea** + **Codex CLI**	Dev Agent in a sandbox → PR for review	TESTED
Edge inference	**MLX**, **llama.cpp**, **Ollama** + open weights (**Qwen**)	Local inference node, token-gated	BUILT
Core & app	**Rust**, **SQLite**, **Flutter**, **Flame**, **Docker**	One shared domain core on every node	BUILT

Mature projects give us **stability and years of hardening**. Our MCP servers give agents **one clean, typed, permissioned interface** to all of them.

PILLAR 1 · CLEAN BY CONSTRUCTION

Standard schemas clean data at the gate, before it reaches any agent




```
// one envelope for every record
{
  "id": "case_01JBX4…",
  "type": "support.case.v1",
  "tenant": "nviai",
  "scope": { "room": "customer-support" },
  "actor": "agent:cskh-agent",
  "version": 7, "op_id": "op_9f2…",
  "source": { "system": "chatwoot",
              "ref": "conv/4821" },
  "links": { "customer": "cus_…",
             "order": "ord_…" },
  "untrusted": ["body"]
}
```

Fixes noise → typed fields · poisoning → provenance · clash → one owner per field · staleness → versions · duplicates → stable IDs · leakage → ACL at retrieval · injection → quarantined text

Talk to agents like colleagues, not tickets



- A **2.5D virtual office**: Lobby with **Mona's** desk, Dev & QA, Product, Sales, Support, HR, Finance, Meeting, Lounge.
- Every agent has an **identity, role, desk and presence**. Presence comes from a real runtime heartbeat, so offline means offline.
- **Walk up and talk**: chat, DM, voice, or pull an agent into a meeting, the same way you would with a human.
- **Room policy = data policy.** What an agent can hear, read and do depends on the rooms it's allowed into.
- Every action leaves an **auditable work history**.

A morning with Mona

09:00 · OPERATOR → MONA (voice, at Mona's desk)
"Stock of SKU-A is low and three customers complained about delays. Take care of it."

MONA → CSKH-AGENT
Drafts replies to 3 open `support.case` records, with cited order status
Linked by stable IDs, so there's no guessing which "ABC Co." it is

MONA → ERP via MCP
Checks stock and proposes a reorder
Approval card shows the exact supplier, quantity and amount

MONA → DEV-AGENT → PROCESS NODE
Fixes the order-tracking bug with Codex in a networkless sandbox and opens a PR

MONA → PM-AGENT
Updates the board and books a 15-min sync in the Meeting room

10:30 · MONA → OPERATOR
Reports back by voice with a summary, evidence links and 1 approval waiting

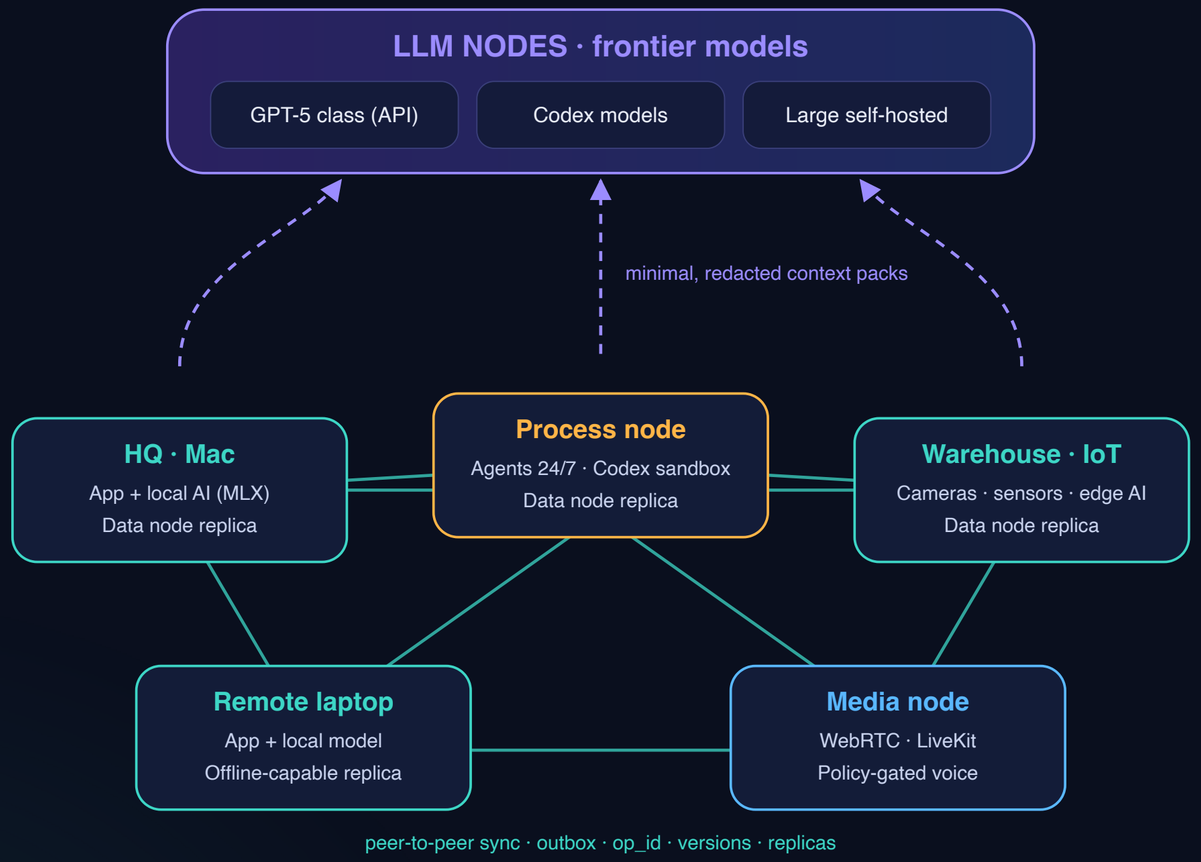
Autonomous, but on a leash

- Scoped agent identity
- Budgets and deadlines
- Durable, fenced job ownership
- Human approval before any side effect
- Cancel anytime · full audit trail

TARGET UX Durable job proposals, opt-in host claims and fenced reports are already built in the prototype.

PILLAR 3 · NVIAI = NEW VISION AI & IOT

Intelligence at the edge, frontier models on demand



- Every site that runs the app runs **its own data node**. Nodes sync **peer to peer with replicas** instead of depending on one central server.
- **Edge AI** on Macs and IoT gateways handles the everyday work: classification, extraction, summaries, vision on cameras and sensors.
- **IoT streams stay local.** Only typed events enter the ledger.
- **LLM nodes** run frontier models that local hardware can't. They're called only with minimal, permission-filtered context.
- Lower server bills, data stays private, and work continues offline.

PILLAR 3 · ROUTING

Send every task to the right model

EDGE · LOCAL MODEL

Fast, private, offline

- Classify & route tickets
- Extract fields into schemas
- Redact PII before escalation
- Summarize meetings & threads
- Camera / sensor events (IoT)

PROCESS NODE · ALWAYS ON

Runs the agents

- 24/7 agent jobs & schedules
- Codex runner in a sandbox
- Workflows & approvals
- Same Rust core as the app
- Scoped service identity

LLM NODE · FRONTIER

Deep reasoning

- Multi-step planning
- Code generation & review
- Long-form drafting
- Hard analysis across departments
- Only with minimal context packs

Routing signals: **data sensitivity · difficulty · latency · cost · connectivity**

MEASURED Qwen3-0.6B-4bit on an Apple M6 Mac: **0.32 s** load · **~352 tok/s** generation · **0.5 GB** peak MLX memory (single synthetic run)

WHAT'S REAL TODAY

A working prototype, measured and not just claimed

~22k
lines of Flutter · Rust · Python

9
domain slices: office, tasks, agents, knowledge, sales, support, HR, finance, ops

~60 fps
office render, p95 build 0.95 ms (idle, debug run)

35
test suites: Rust, HTTP, Flutter, macOS integration

Built

Seeded 2.5D office with pathfinding, presence and room chat · server-enforced room access · task board + persistent outbox · durable agent jobs with fenced claims · room-scoped docs, search, meetings, whiteboard · CRM, contracts, bookings, support cases · leave, expenses, journals, payment authorizations, incidents · local MLX inference

Tested in isolation

LiveKit room-epoch isolation (synthetic audio) · Codex CLI 0.164 in a networkless, read-only, non-root container with watchdog · SQLite backup & restore

Not yet

Voice in the app · real Codex provider & PR flow · E2EE sync · remote deployment · production hardening

Our rule: **if we didn't measure it, we don't claim it.** Seeded agents are shown as offline until a real runtime is running.

BUILT WITH CODEX, AND CODEX BUILT IN

Codex is both how we build it and how its agents work

How we build with Codex

1 · SPEC
Reference docs: product, architecture, office, security, seed, open decisions

2 · STAGE PROMPTS
Codex builds vertical slices end to end: Rust core → node → Flutter UI → tests

3 · VERIFY
Codex has to run, measure and report real results, and label anything fake

4 · ITERATE
Capability matrix + ADRs keep each stage honest

Codex as the Dev Agent's engine

- Process node claims an **approved job** (durable, fenced)
- Runs `codex exec` in a **networkless, read-only, non-root** container
- PID / memory / CPU limits, deadline, watchdog, cancel
- Output becomes a **PR for human review**, never a silent merge
- Agent identity stays separate from the coding engine, so engines can be swapped

ROADMAP

Each stage ships as a usable vertical slice

01

Virtual office & comms

Rooms, identities, presence, chat, voice, policies

PROTOTYPE

02

Tasks & workflow

Projects, boards, assignment to people or agents, audit

PROTOTYPE

03

Mona & Dev Agent

Agent runtime, approvals, Codex runner, MCP gateway

PARTIAL

04

Knowledge & edge AI

Docs, search, meetings, whiteboard, local/cloud inference

PARTIAL

05

Customers & sales

CRM, support, contracts, bookings, integrations

PARTIAL

06

HR, finance & ops

Leave, expenses, journals, payments, incidents, separation of duties

PARTIAL

07

Hardening & platforms

Security review, backup, E2EE, IoT nodes, more platforms

NEXT

macOS first. Every stage ships with **permission checks, tests and documentation**.

WHY MONASTACK

Stop rebuilding apps for humans. Build the company for agents.

AI-NATIVE PRODUCTS & OPS

AI-first from the schema up, with one clean data layer for every department.

AUTONOMOUS & ADAPTIVE

Agents work 24/7 on process nodes, with fenced jobs and human approvals.

DEEP DOMAIN

Real business rules for support, sales, HR and finance, including separation of duties.

Thank you. **Team NVIAI** · New Vision AI & IoT · nviai.com